CAPSTONE PROJECT

Explainable Federated Learning Framework for Predicting Customer Churn in Telecom Networks

IILM University, Greater Noida

(Project Members)

1) Gaurav Upadhyay (CS-2341884)
2) Kritik Kumar (CS-23411627)
3) Avneesh Kumar (CS-2341418)
4) Akshma Pundir (CS-23411618)

B.Tech CSE 4th Year
Batch : 2023 -2027

Under the Guidance of

Dr. Sumit Kumar

Phase II Progress Report

Status: Working prototype executed; centralized-model SHAP explanations and a validation-based threshold experiment completed; robustness and federated-model explainability remain pending

1. Executive Summary

This project studies telecom customer churn prediction using simulated Federated Learning (FL), with Explainable AI (XAI) intended to explain model predictions. The Phase I research foundation includes the problem statement, objectives, expected outcomes, overall research gap, and a literature survey matrix.

An initial Python prototype has now been run successfully on the IBM Telco Customer Churn CSV. It loads 7,043 rows and 21 original columns, preprocesses the data, trains a centralized Logistic Regression baseline, simulates three FL clients using a custom NumPy FedAvg-style implementation, evaluates both models, and saves model comparison, confusion matrices, permutation feature importance, and one example prediction.

Actual results: centralized accuracy 0.7381, precision 0.5043, recall 0.7834, F1 0.6136, ROC-AUC 0.8413. Federated accuracy 0.7786, precision 0.7422, recall 0.2540, F1 0.3785, ROC-AUC 0.8363. The higher federated accuracy does not mean it is the stronger churn detector; its recall and F1 are low at the default 0.5 threshold.

2. Project Definition and Research Gap

2.1 Problem statement

Customer churn can reduce telecom revenue and damage long-term customer relationships. Centralized ML can predict churn but requires bringing data to a common location. Telecom data may be distributed and sensitive. FL allows clients to train collaboratively while keeping raw records local. Yet predictive performance and data locality do not by themselves explain why a customer is predicted to churn. This project combines FL and XAI and compares the federated approach with centralized learning.

2.2 Core research question

How can telecom customer churn be predicted using decentralized customer data while keeping the data local and still providing understandable explanations for the model's predictions?

2.3 Objectives

1. Study ML, deep learning, FL, and XAI approaches for telecom churn prediction.
2. Develop a framework where simulated clients train collaboratively while keeping raw training records local.
3. Use SHAP and/or LIME to identify factors influencing churn predictions.
4. Evaluate accuracy, precision, recall, F1-score, and ROC-AUC.
5. Compare centralized and federated approaches in performance, data locality, and explainability.
6. Analyse whether explanations can support understandable customer-retention insights.

2.4 Research gap from the project brief

The stated gap is that explainable churn prediction is commonly studied with centralized data, while FL churn studies tend to emphasize prediction and data locality without sufficient attention to explaining federated predictions. The proposed contribution is to combine decentralized collaborative training with explanations and experimentally compare centralized and federated approaches.

A literature survey matrix exists as an Excel file shared in the conversation. The matrix was not independently inspected for this report, so paper-specific statements should be checked against it before final submission.

3. Dataset

Dataset used in the successful run: IBM Telco Customer Churn CSV. The script downloaded it from:

https://raw.githubusercontent.com/IBM/telco-customer-churn-on-icp4d/master/data/Telco-Customer-Churn.csv

Property	Value
Rows	7,043
Original columns	21
Predictors before encoding	19 original attributes after removing customerID and separating Churn
Target	Churn: Yes → 1; No → 0
Target distribution printed	No churn 73.5%; churn 26.5% (rounded)
Split	80% train / 20% test; stratified; random_state=42
Training records	5,634
Test records	1,409
Provenance	Public experimental/demo dataset. The source used here does not establish that all records are verified live customer data or definitively synthetic.

3.1 Original 21 attributes

Attribute	Meaning / handling
customerID	Unique customer identifier; removed before training.
gender	Customer gender.
SeniorCitizen	Senior-citizen indicator.
Partner	Whether the customer has a partner.
Dependents	Whether the customer has dependents.
tenure	Number of months with the company.
PhoneService	Phone service subscription.
MultipleLines	Multiple phone-line status.
InternetService	Internet-service type.
OnlineSecurity	Online-security subscription.
OnlineBackup	Online-backup subscription.
DeviceProtection	Device-protection subscription.
TechSupport	Technical-support subscription.
StreamingTV	Streaming-TV subscription.
StreamingMovies	Streaming-movies subscription.
Contract	Contract type.
PaperlessBilling	Paperless-billing status.
PaymentMethod	Payment method.
MonthlyCharges	Monthly service charges.
TotalCharges	Total accumulated charges; converted to numeric, invalid/blank values coerced to missing.
Churn	Target label; Yes=1, No=0.

4. Environment and Project Files

- Environment: Windows, VS Code integrated PowerShell terminal; Python 3.11 is indicated by the downloaded wheel tags.
- Virtual environment: .venv; terminal prompt confirmed it was active.
- requirements.txt lists pandas, numpy, scikit-learn, matplotlib, joblib, streamlit, and shap.
- train.py: main experiment script; successfully run with command `python train.py`.
- app.py: Streamlit dashboard run is shown in supplied screenshot evidence (Streamlit_run.pdf); a separate clean-environment reproduction has not been reported.
- README.md: setup and demonstration notes.
- Generated results are saved under outputs/.

5. Preprocessing and Data Handling

- Loader checks a provided --data path, then data/Telco-Customer-Churn.csv and Telco-Customer-Churn.csv; if absent, it downloads the public CSV and saves it in data/.
- Duplicate rows are dropped.
- Churn strings are stripped and mapped Yes → 1 and No → 0; rows with missing target after mapping are dropped.
- customerID is removed from model inputs.
- TotalCharges is converted using pandas.to_numeric(errors='coerce').
- train_test_split uses test_size=0.2, random_state=42, and stratify=y.
- Numeric columns: median imputation, then StandardScaler.
- Categorical columns: most-frequent imputation, then OneHotEncoder(handle_unknown='ignore').
- A ColumnTransformer applies both pipelines. The transformed sparse matrices are converted to dense arrays for the NumPy federated model.

5.1 Preprocessing limitation

The preprocessor is fitted on the overall training split before transformed training data are partitioned into simulated clients. This is a prototype limitation: it does not fully simulate independent client-side preprocessing. Raw records are not passed between clients in the training loop, but this preprocessing design should be documented and improved for a stronger experiment.

6. Centralized Baseline

Model: scikit-learn LogisticRegression(max_iter=1500, class_weight='balanced', random_state=42). It is trained on the transformed training set. Test probabilities use predict_proba(X_test)[:, 1]. Predicted labels use a default threshold of 0.5.

Metric	Value	Interpretation
Accuracy	0.7381 (73.81%)	Fraction of test records classified correctly.
Precision	0.5043 (50.43%)	Among predicted churners, fraction that actually churned.
Recall	0.7834 (78.34%)	Among actual churners, fraction identified.
F1-score	0.6136 (61.36%)	Harmonic mean of churn precision and recall.
ROC-AUC	0.8413 (84.13%)	Probability-ranking/discrimination metric.

6.1 Centralized classification report

Class	Precision	Recall	F1	Support
No churn	0.90	0.72	0.80	1,035
Churn	0.50	0.78	0.61	374
Accuracy	—	—	0.74	1,409
Macro avg	0.70	0.75	0.71	1,409
Weighted avg	0.80	0.74	0.75	1,409

7. Federated Learning Implementation

A lightweight custom NumPy FedAvg-style binary logistic regression was implemented. Three clients are simulated as partitions of one public dataset on one computer; they are not real telecom companies.

Parameter / mechanism	Implementation detail
Clients	3
Federated rounds	12
Local epochs / gradient steps per round	3
Learning rate	0.08
Random seed	42
Initialization	Global weights and bias start at zero.
Client partition	Training indices shuffled with numpy.random.default_rng(seed=42), then divided with numpy.array_split into three clients.
Local model	Binary logistic regression optimized with gradient descent.
Sigmoid stability	Logits clipped to [-35, 35].
Gradient	Mean binary cross-entropy gradient; weight gradient adds 0.001 × weights as mild L2 regularization.
Client update	Each client starts from current global weights/bias, performs three local gradient steps and returns parameters.
Aggregation	Weights and bias averaged in proportion to each client's record count (FedAvg-style weighted average).
Logged rounds	Training log-loss printed at rounds 1, 3, 6 and 12.
Test predictions	Global model sigmoid probabilities; class threshold 0.5.

7.1 Observed global training loss

Round	Log-loss
1/12	0.6112
3/12	0.5423
6/12	0.4987
12/12	0.4603

The logged training loss decreased across the shown checkpoints, indicating optimization progress on the training data. This does not, by itself, prove good generalization or high churn recall.

8. Results: Centralized vs Federated

Metric	Centralized Logistic Regression	Federated Logistic Regression (FedAvg)
Accuracy	0.7381	0.7786
Precision	0.5043	0.7422
Recall	0.7834	0.2540
F1-score	0.6136	0.3785
ROC-AUC	0.8413	0.8363

8.1 Interpretation of actual results

- Federated accuracy is higher (77.86% vs 73.81%) and precision is higher (74.22% vs 50.43%) at threshold 0.5.
- Federated recall is much lower (25.40% vs 78.34%) and F1 is lower (37.85% vs 61.36%); it misses many actual churners at this threshold.
- ROC-AUC values are close: 0.8363 federated versus 0.8413 centralized. Threshold-specific predictions differ despite similar ranking ability.
- These numbers describe this implementation, partition, seed, optimization setup, and threshold; they do not prove FL is universally better or worse.
- Next investigate confusion matrices, probability distributions, validation-based threshold selection, class imbalance, optimization convergence, and multiple random seeds.

9. Explainability Implemented So Far

- Permutation feature importance is calculated for the centralized Logistic Regression model using sklearn.inspection.permutation_importance.
- Parameters: scoring='f1', n_repeats=5, random_state=42, n_jobs=-1.
- Each transformed feature is shuffled and the change in F1 is measured; output includes mean importance and standard deviation.
- Feature names are retrieved using preprocessor.get_feature_names_out() where available.
- outputs/feature_importance.csv stores the table; outputs/feature_importance.png plots the top 15 features ranked by mean importance.
- Permutation feature importance is complemented by centralized-model SHAP global and individual explanations, generated in the later successful train_with_shap.py run documented in Section 16.
- The later SHAP output ranks tenure, MonthlyCharges, and TotalCharges among the largest mean absolute SHAP contributions; see Section 16.3.

9.1 Example prediction

Field	Value
Actual churn label	0 (No churn)
Centralized probability	0.115472
Centralized prediction	0 (No churn)
Federated probability	0.121078
Federated prediction	0 (No churn)

This is one illustrative test example, not evidence of overall performance. The script saves the probabilities and labels and a later run generated an individual SHAP explanation for a test example; see Section 16.4.

10. Generated Artifacts

File	Status / contents
outputs/model_comparison.csv	Generated; both models and five metrics.
outputs/confusion_matrices.png	Generated; side-by-side confusion matrices.
outputs/feature_importance.csv	Generated; permutation-importance mean and standard deviation.
outputs/feature_importance.png	Generated; top 15 transformed features by permutation importance.
outputs/example_prediction.csv	Generated; actual label and both models' probabilities/predictions for one test example.
app.py	Streamlit dashboard code created; execution not yet confirmed.
README.md	Setup and demonstration instructions created.

11. Limitations and Research Cautions

- Dataset provenance: public experimental/demo dataset; do not claim verified live customer data or definitively synthetic data without authoritative evidence.
- Clients are simulated from one dataset on one machine; no independent telecom organizations participate.
- Model updates alone do not guarantee privacy. No secure aggregation, differential privacy, or formal privacy audit is implemented.
- Preprocessing is fitted before the client split, so client independence is not fully represented.
- The centralized sklearn model and custom gradient-descent FedAvg model use different optimization implementations/settings; the comparison is not a perfectly controlled algorithm-only comparison.

- The original comparison uses a fixed 0.5 threshold. A separate validation-based threshold experiment was subsequently completed on a new train/validation/test split.
- Only one train/test split and one random seed are reported; repeated runs and confidence intervals are pending.
- Centralized-model SHAP explanations are implemented. SHAP/LIME explanations for the federated model remain pending.
- No real retention intervention or business-impact evaluation has been performed.

12. Completion Status

Component	Status	Evidence / next step
Problem statement, objectives, research gap	Research foundation prepared	Project brief supplied.
Literature review	Completed	Literature survey matrix exists; verify individual citations for final report.
Dataset loading	Completed	7,043 rows, 21 columns loaded.
Preprocessing and split	Implemented and run	Stratified 80/20 split; pipeline described above.
Centralized baseline	Implemented and run	Metrics and classification report captured.
Three-client FedAvg	Implemented and run	12 rounds; loss checkpoints and test metrics captured.
Model comparison	Implemented and run	CSV and confusion-matrix plot generated.
Initial XAI	Implemented and run	Permutation importance CSV and plot generated.
SHAP / LIME	Pending	Add global and individual explanations.
Streamlit interface	Code created; run unconfirmed	Run `streamlit run app.py` after checking outputs.
Non-IID and repeated-seed experiments	Pending	Assess heterogeneity and robustness.
Final report and evidence form	Pending	Include screenshots, plots, results, discussion and limitations.

13. Recommended Next Steps

7. Preserve the output CSV/PNG files and take screenshots of the successful terminal run.
8. Open outputs/feature_importance.csv to identify the actual top features; do not infer them without checking.
9. Create a validation split within training data and tune any decision threshold on validation data only; leave test data untouched for final evaluation.
10. Investigate the federated model's low churn recall and F1; document confusion matrices and compare training convergence.
11. Integrate SHAP for global feature impact and at least one individual prediction; clearly describe how explanations are computed.
12. Run repeated random seeds and, if time permits, compare IID and deliberately non-IID client partitions.
13. Document simulated-client scope and privacy limitations accurately.
14. Rehearse: dataset → preprocessing → centralized model → local client training → FedAvg aggregation → comparison → feature importance → limitations and planned work.

14. Suggested Mentor Summary

"I have completed the literature review and implemented an initial runnable prototype. It uses 7,043 records from the IBM Telco Customer Churn dataset. I trained a centralized Logistic Regression baseline and a custom three-client FedAvg-style logistic model. At the 0.5 threshold, centralized recall and F1 are 78.34% and 61.36%, while federated recall and F1 are 25.40% and 37.85%. Although federated accuracy is higher, its recall is low, so the next work is to investigate optimization and threshold selection. I have also generated permutation feature-importance outputs. Centralized-model SHAP explanations and a separate validation-based threshold experiment have been completed. Repeated-seed validation, federated-model explanations, and further privacy-aware design remain in progress."

15. Reproducibility

The reported run was executed in a Windows VS Code PowerShell terminal with the .venv environment active, using `python train.py`. The terminal confirmed dataset download and output-file generation. Seed 42 is used for the split and specified random operations. Exact installed package versions were not saved as a lock file; for reproducibility, run `python -m pip freeze > requirements-lock.txt` in the activated environment.

End of report

16. Phase II Update: SHAP Run and Output Verification

Update basis: the successful terminal output and the CSV/PNG files supplied after the initial report was prepared. This section supplements the original report; it does not replace the original methodology or earlier limitations.

16.1 Execution status

The student ran train_with_shap.py successfully. The terminal output showed the dataset loading, centralized Logistic Regression training, the three-client FedAvg-style simulation (12 rounds), model comparison, permutation feature importance, and SHAP explanation stage completing without a reported traceback. The terminal listed the global SHAP importance CSV/plot and individual SHAP explanation CSV/plot as generated. The supplied CSV contents were inspected for consistency with the terminal output. This is evidence of a successful run in the student's environment; the run has not been independently reproduced in a separate environment.

16.2 Model comparison from the supplied CSV

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Centralized Logistic Regression	0.7381	0.5043	0.7834	0.6136	0.8413
Federated Logistic Regression (FedAvg)	0.7786	0.7422	0.2540	0.3785	0.8363

Interpretation: at the fixed 0.5 threshold, the federated simulation has higher accuracy and precision, but lower recall and F1 than the centralized baseline. The values do not support a claim that the federated model is better overall. The comparison remains limited because the centralized model uses scikit-learn LogisticRegression with class_weight='balanced', while the custom federated gradient-descent implementation does not use an equivalent class-weighting setup.

16.3 Global SHAP importance

The global plot ranks transformed features by mean absolute SHAP value for the centralized model. Values are on the model output's log-odds scale; mean absolute values show average contribution magnitude, not direction or causation.

Transformed feature	Mean absolute SHAP value
num__tenure	0.995347
num__MonthlyCharges	0.567263
num__TotalCharges	0.362923
cat__InternetService_Fiber optic	0.348088
cat__Contract_Month-to-month	0.323959
cat__Contract_Two year	0.284380
cat__InternetService_DSL	0.283293
cat__StreamingMovies_Yes	0.129269
cat__PaperlessBilling_No	0.125604
cat__StreamingTV_Yes	0.121047

In this run, tenure has the largest mean absolute SHAP value (0.995347), followed by MonthlyCharges (0.567263) and TotalCharges (0.362923). InternetService_Fiber optic (0.348088), Contract_Month-to-month (0.323959), Contract_Two year (0.284380), and InternetService_DSL (0.283293) are also among the higher-ranked features. These results describe model contributions in this dataset and setup; they do not establish that the features cause churn.

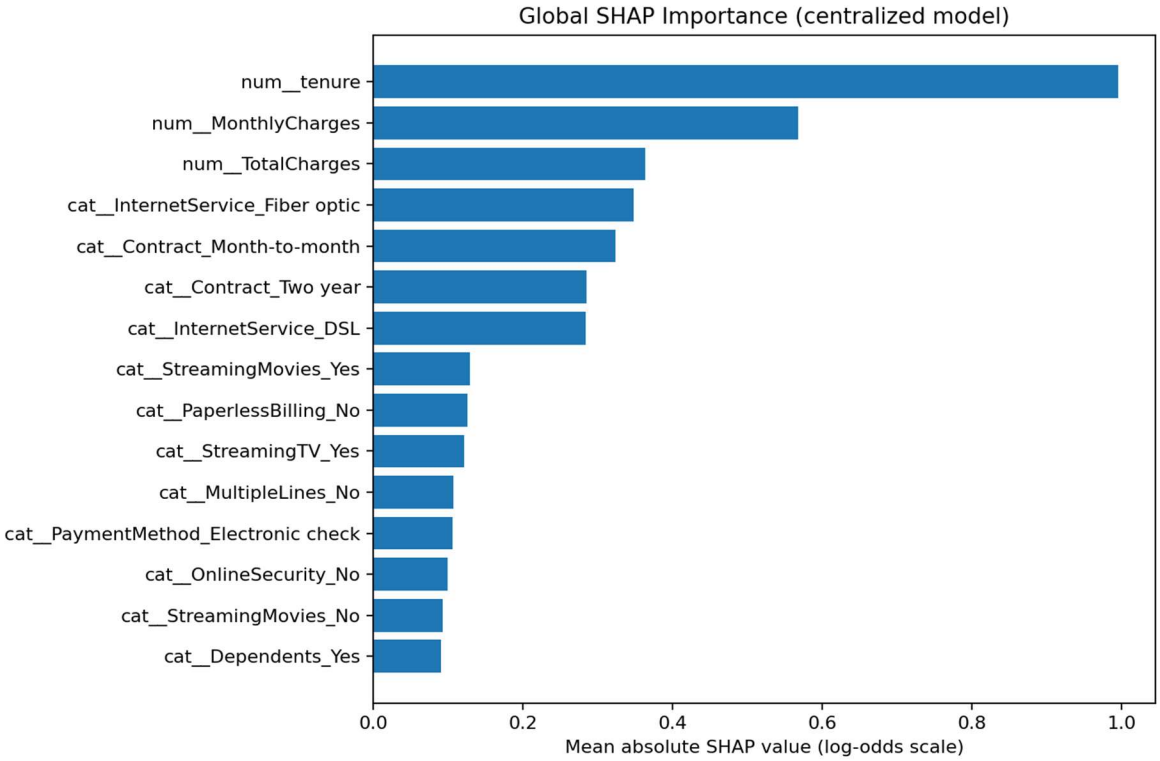


Figure 1. Global SHAP importance for the centralized model (supplied output).

16.4 Individual SHAP explanation

The supplied individual SHAP CSV describes one test example. The original example-prediction output reports actual_churn = 0 (no churn) and centralized churn probability = 0.115472 (approximately 11.55%), with centralized prediction = 0. The individual SHAP values explain contributions to the centralized model output on the log-odds scale.

Transformed feature	SHAP value	Direction in model output
num__tenure	-1.916817	Away from churn
num__TotalCharges	1.272664	Toward churn
num__MonthlyCharges	-1.059282	Away from churn
cat__Contract_Two year	-0.593755	Away from churn
cat__Contract_Month-to-month	-0.411307	Away from churn
cat__InternetService_Fiber optic	0.387935	Toward churn
cat__InternetService_DSL	0.223563	Toward churn
cat__StreamingMovies_Yes	0.170487	Toward churn
cat__StreamingTV_Yes	0.167704	Toward churn
cat__Dependents_Yes	-0.140099	Away from churn

For this example, tenure has a strong negative SHAP value (-1.916817), pushing the model output away from churn; TotalCharges has a positive value (+1.272664), pushing toward churn; MonthlyCharges has a negative value (-1.059282), and the Two-year contract indicator has a negative value (-0.593755). These are contributions to log-odds, not percentage-point changes in probability. The plot is one example and is not evidence of overall model performance.

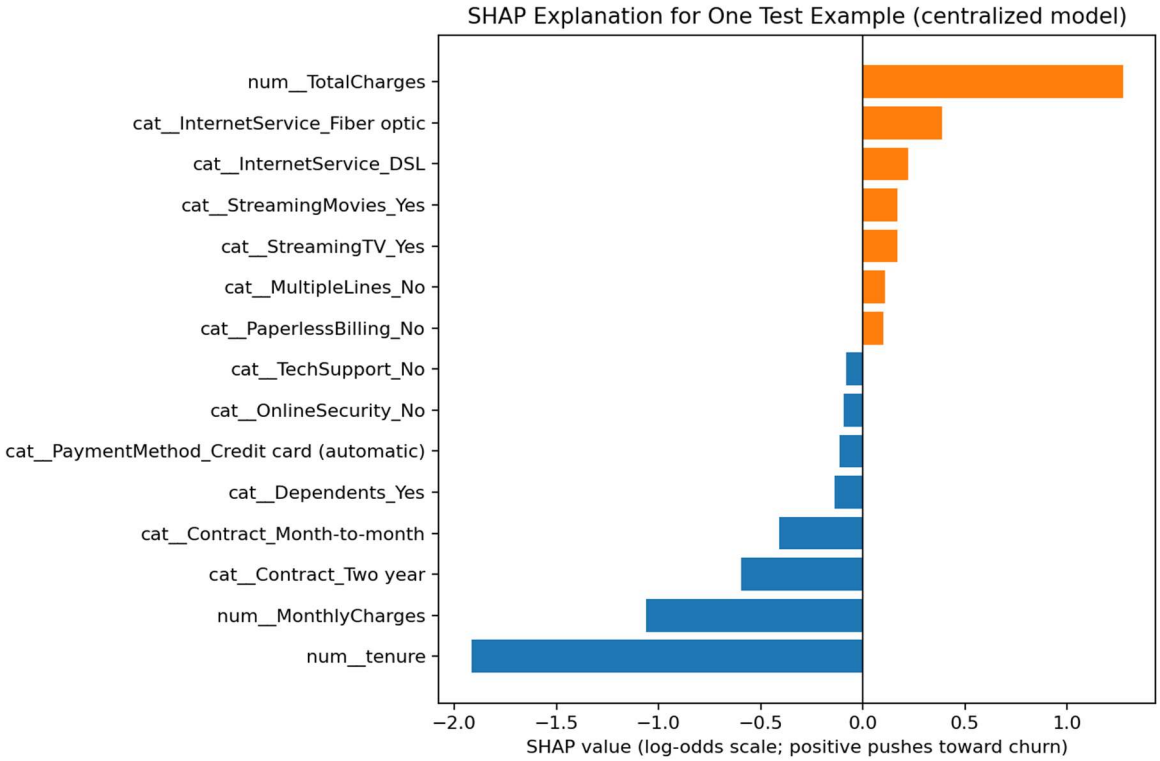


Figure 2. SHAP explanation for one test example from the centralized model (supplied output).

16.5 Updated implementation status and limitations

- Completed in the reported run: centralized model evaluation, three-client FedAvg-style simulation, model comparison output, permutation feature importance, global SHAP importance for the centralized model, and SHAP values for one centralized-model test example.
- Not completed: SHAP/LIME explanations for the federated model; threshold selection using a validation set; repeated-seed robustness experiments; and controlled comparisons that align class weighting and optimization choices.
- The clients remain simulated partitions of one dataset on one computer; they are not independent telecom organizations.
- The preprocessing transformer is fitted on the complete training split before the simulated client partition, so fully independent client-side preprocessing is not demonstrated.
- No secure aggregation, differential privacy, or formal privacy audit is implemented. Keeping raw rows out of the model-update loop alone does not guarantee privacy.
- The dataset source does not establish that the records are verified live customer data or definitively synthetic. Avoid making either unsupported claim.
- The supplied CSVs and plots were inspected for contents, but the experiment was not independently reproduced outside the student's environment.

17. Phase III Update: Validation-Based Threshold Experiment

Update basis: The results below are taken from the successful terminal output of threshold_validation_experiment.py. This is a separate experiment using a new data split; it does not replace the original results in Sections 6–8 and 16.

17.1 Experimental design

- The dataset contains 7,043 records. A stratified three-way split was used: 64% training (4,507 records), 16% validation (1,127 records), and 20% final test (1,409 records).
- The preprocessing pipeline was fitted on the training partition only, then applied to validation and final test data.
- Both the centralized Logistic Regression model and the custom three-client FedAvg-style model were evaluated at threshold 0.50 and at a model-specific threshold selected by maximizing F1 on the validation partition.
- The selected thresholds were 0.62 for the centralized model and 0.33 for FedAvg. Final test metrics were calculated on the separate final test partition.
- Because this experiment uses a different split from the earlier 80/20 experiment, results across the two splits should not be interpreted as a direct before-and-after comparison.

17.2 Final test results

Model / threshold	Accuracy	Precision	Recall	F1-score	ROC-AUC
Centralized, threshold 0.50	0.7417 (74.17%)	0.5086 (50.86%)	0.7888 (78.88%)	0.6184 (61.84%)	0.8421 (84.21%)
Centralized, validation-tuned 0.62	0.7708 (77.08%)	0.5544 (55.44%)	0.6952 (69.52%)	0.6168 (61.68%)	0.8421 (84.21%)
FedAvg, threshold 0.50	0.7771 (77.71%)	0.7344 (73.44%)	0.2513 (25.13%)	0.3745 (37.45%)	0.8362 (83.62%)
FedAvg, validation-tuned 0.33	0.7523 (75.23%)	0.5234 (52.34%)	0.7487 (74.87%)	0.6161 (61.61%)	0.8362 (83.62%)

17.3 Interpretation

- For FedAvg on this experiment's final test set, changing the threshold from 0.50 to the validation-selected 0.33 increased recall from 25.13% to 74.87% and F1-score from 37.45% to 61.61%.
- FedAvg precision decreased from 73.44% to 52.34%, while accuracy decreased from 77.71% to 75.23%. This illustrates a trade-off between identifying more churners and producing more false-positive churn predictions.
- For the centralized model, the selected threshold of 0.62 increased accuracy from 74.17% to 77.08% and precision from 50.86% to 55.44%, while recall decreased from 78.88% to 69.52%; F1 changed slightly from 61.84% to 61.68%.
- ROC-AUC remains unchanged for each model across its two thresholds because threshold selection changes class labels, not the underlying probability ranking.
- These results indicate that FedAvg recall at threshold 0.50 is sensitive to threshold choice. They do not establish that FL is universally superior or that these thresholds are optimal for real-world retention decisions.

17.4 Generated evidence files

File	Purpose
threshold_experiment_outputs/central_validation_thresholds.csv	Centralized validation metrics across candidate thresholds and selected threshold.
threshold_experiment_outputs/fedavg_validation_thresholds.csv	FedAvg validation metrics across candidate thresholds and selected threshold.
threshold_experiment_outputs/test_threshold_comparison.csv	Final test metrics at threshold 0.50 and the validation-selected threshold.
threshold_experiment_outputs/test_confusion_matrices.csv	Final test confusion-matrix counts for evaluated model/threshold combinations.

17.5 Remaining limitations and work

- Only one train/validation/test split and one run are reported; repeated seeds, confidence intervals, and variability have not been evaluated.
- Threshold selection maximizes F1 on the validation partition. A production threshold should also consider business costs, retention capacity, and the relative costs of false positives and false negatives.
- The three federated clients are simulated partitions of one public dataset on one computer; they are not independent telecom organizations.
- The centralized and custom federated implementations differ in optimization and class-weighting choices, so this is not a perfectly controlled algorithm-only comparison.
- The threshold experiment fits preprocessing on the training partition only. The original experiment described earlier fitted preprocessing before partitioning training records into simulated clients; that limitation remains relevant to the original results.

- SHAP explanations are implemented for the centralized model only. Federated-model explanations, repeated-seed robustness testing, non-IID client experiments, secure aggregation, differential privacy, and formal privacy evaluation remain future work.

17.6 Updated implementation status

Component	Current status	Evidence / note
Dataset loading and preprocessing	Implemented and run	Public IBM Telco Customer Churn CSV; 7,043 rows and 21 original columns.
Centralized Logistic Regression	Implemented and evaluated	Original fixed-threshold results and separate validation-threshold experiment reported.
Three-client FedAvg-style model	Implemented and evaluated	Custom NumPy implementation; clients are simulated.
Permutation feature importance	Generated	CSV and plot saved.
Centralized-model SHAP	Generated	Global importance and one individual test-example explanation saved.
Threshold selection	Completed for one split	Thresholds 0.62 (centralized) and 0.33 (FedAvg), selected using validation F1.
Streamlit dashboard	Run shown in supplied screenshot evidence	Streamlit_run.pdf; separate clean-environment reproduction not reported.
Repeated-seed and non-IID experiments	Pending	Needed to assess robustness and client heterogeneity.
Federated-model explanations and privacy mechanisms	Pending	No secure aggregation, differential privacy, or formal privacy audit implemented.

18. Updated Mentor Summary

The project has a working prototype for telecom churn prediction using a centralized Logistic Regression baseline and a custom three-client FedAvg-style model on the public IBM Telco Customer Churn dataset. Centralized-model permutation importance and SHAP global and individual explanations have been generated, and the Streamlit dashboard is shown running in the supplied screenshot evidence. A separate validation-based threshold experiment used a 64%/16%/20% train/validation/test split, selecting thresholds of 0.62 for the centralized model and 0.33 for FedAvg. On that experiment's final test set, FedAvg recall increased from 25.13% at threshold 0.50 to 74.87% at threshold 0.33, while precision decreased from 73.44% to 52.34%. The experiment remains preliminary: only one split is reported, clients are simulated, model optimization choices differ, and federated-model explainability and privacy mechanisms remain future work.

19. Evidence and Reproducibility Notes

- Original experiment: train_with_shap.py; earlier results remain documented in Sections 6–8 and 16 and must remain distinct from the threshold experiment.
- Threshold experiment: threshold_validation_experiment.py; the reported terminal output confirms the 4,507 / 1,127 / 1,409 split, selected thresholds, and final test metrics in Section 17.
- Dashboard evidence: Streamlit_run.pdf contains screenshots of the dashboard interface and displayed evaluation/explainability outputs. A separate clean-install reproduction is not claimed.
- For submission, include this updated report and the four threshold experiment CSV files. Keep the SHAP/feature-importance outputs and dashboard screenshot PDF.
- Before submission, verify the university form's project ID/title, guide name and email, minimum three work-progress records, both required evaluation sections, and access permissions for repository and Drive links. These administrative items are not verified by this technical report.